

Configuración inicial de un servidor Linux

Acceso, administración, identidad, red y enrutamiento

✉ José Domingo Muñoz

🏫 IES Gonzalo Nazareno · Dos Hermanas

📖 SRI · Servicios de Red e Internet

01

Acceso seguro al servidor

SSH con autenticación por clave pública

¿Qué es SSH?

SSH (*Secure Shell*) es el protocolo estándar para administrar equipos Linux de forma remota a través de un **canal cifrado**.

- Sustituye a protocolos antiguos sin cifrar como **Telnet** o **rsh**
- Cifra tanto las credenciales como todo el tráfico de la sesión
- El cliente (`ssh`) se conecta al demonio `sshd` del servidor, por el puerto **TCP 22**
- La configuración del servidor está en `/etc/ssh/sshd_config`
- Además de una shell remota, permite transferir ficheros (`scp` , `sftp`) y crear túneles

 El canal ya viaja cifrado por defecto. Lo que queda por decidir es **cómo se demuestra la identidad**: con contraseña o con clave pública.

¿Por qué clave pública en lugar de contraseña?

La autenticación por contraseña es vulnerable a **ataques de fuerza bruta** y depende de que el usuario elija y custodie un secreto fuerte. La clave pública sustituye ese secreto por un **par criptográfico**: el servidor verifica la identidad sin que ninguna contraseña viaje por la red.

VENTAJAS FRENTE A LA CONTRASEÑA

- **Mayor seguridad**: no se transmite ningún secreto en la conexión
- **Resistencia a fuerza bruta** y a ataques de *sniffing*
- **Automatización**: scripts y tareas remotas sin intervención humana
- **Comodidad**: un único par de claves vale para muchos servidores

Cifrado simétrico vs. asimétrico

SIMÉTRICO

- Una **única clave** cifra y descifra
- Rápido: se usa para cifrar grandes volúmenes de datos
- Es lo que usa el propio **canal SSH** para cifrar el tráfico, una vez establecida la conexión

ASIMÉTRICO

- Un **par de claves** matemáticamente vinculado
- Lo que cifra una clave, solo lo descifra la otra
- Más lento: se reserva para la fase de **autenticación de identidad**

Firma digital y reto criptográfico

- Cifrar con la clave **pública** de alguien: solo esa persona puede descifrarlo, con su clave **privada**. Sirve para que solo el destinatario pueda leer el mensaje.
- Cifrar con mi propia clave **privada**: a esto se le llama **firmar**. Cualquiera puede descifrarlo con mi clave **pública** — eso es **verificar la firma**. Como solo yo he podido cifrarlo con mi privada, demuestra que el mensaje es mío.
- El **reto criptográfico**: un dato que el servidor envía para que el cliente lo firme (lo cifre con su clave privada) en el momento. El servidor lo verifica descifrándolo con la clave pública del cliente.



Así se demuestra que el cliente posee la clave privada, **sin que esta viaje nunca por la red**.

Criptografía asimétrica: la idea fundamental

" Un par de claves matemáticamente vinculadas: lo cifrado con una sólo puede verificarse con la otra.

CLAVE PRIVADA

- **Permanece en el cliente**, nunca se comparte
- Se protege con permisos estrictos del sistema de ficheros
- Puede ir cifrada con una *passphrase* adicional
- Es la prueba de identidad del usuario

CLAVE PÚBLICA

- **Se instala en el servidor**, en cada cuenta autorizada
- Puede distribuirse libremente: no compromete la privada
- El servidor la usa para **verificar** al cliente
- Sólo autentica a quien posea la privada correspondiente

¿Cómo se establece la conexión?

1. El cliente solicita conectarse con un usuario del servidor
2. El servidor consulta las **claves públicas autorizadas** para esa cuenta
3. Lanza un **reto criptográfico** que sólo puede resolver quien tenga la clave privada
4. El cliente responde firmando el reto con su clave privada
5. El servidor verifica la firma con la clave pública y **autoriza el acceso**

 El secreto (la clave privada) **nunca abandona el cliente**. El servidor sólo necesita la pública para verificar.

Herramientas en la práctica

SSH-KEYGEN

- Genera el **par de claves** (privada y pública) en el cliente
- Se guarda por defecto en `~/.ssh/` (`id_rsa`, `id_ed25519` ...)
- Permite proteger la clave privada con una **passphrase**

SSH-COPY-ID

- Copia automáticamente la **clave pública** del cliente al servidor
- La añade al archivo `~/.ssh/authorized_keys` del usuario remoto
- Alternativa manual: copiar la clave pública a `~/.ssh/authorized_keys` en el servidor



El archivo `~/.ssh/authorized_keys` contiene **todas las claves públicas** que pueden iniciar sesión en esa cuenta.

02

Administración delegada

Privilegios controlados con sudo

El problema del usuario root

Trabajar directamente como **root** rompe el principio de **mínimo privilegio**: un error tipográfico puede dañar el sistema, y todas las acciones quedan bajo un único identificador, dificultando la auditoría.

¿QUÉ APORTA **SUDO**?

- Permite que **usuarios concretos** ejecuten tareas administrativas con sus propias credenciales
- **No es necesario compartir** la contraseña de root
- Cada acción privilegiada queda **registrada con el usuario real** que la ejecuta
- Se puede limitar **qué comandos** puede ejecutar cada usuario

Ventajas frente a iniciar sesión como root

AUDITORÍA Y TRAZABILIDAD

- Cada comando privilegiado queda registrado con el usuario que lo lanzó
- Permite reconstruir quién hizo qué y cuándo
- Útil en equipos con varios administradores

CONTROL GRANULAR

- Se puede autorizar a un usuario sólo a comandos concretos
- Se decide si requiere o no contraseña
- Se asignan permisos por **usuario** o por **grupo**
- Aplica el principio de **mínimo privilegio**

Configuración: sudoers y sudoers.d

/ETC/SUDOERS

- Archivo principal de configuración de sudo
- Define qué usuarios/grupos pueden ejecutar qué comandos con privilegios
- Se edita con **visudo**, que valida la sintaxis antes de guardar

/ETC/SUDOERS.D/

- Directorio con archivos **modulares** de configuración
- Cada archivo añade reglas sin tocar el principal
- Facilita la gestión por **automatización** (Ansible, paquetes, scripts)



Usar siempre **visudo**: un error en sudoers puede dejar el sistema sin acceso administrativo.

Comandos habituales con sudo

HABILITAR A UN USUARIO

```
# Añadir al grupo sudo
sudo usermod -aG sudo nombre_usuario

# Editar la configuración con validación
sudo visudo
```

`visudo` abre `/etc/sudoers` en un editor y **comprueba la sintaxis** antes de guardar.

EJEMPLOS DENTRO DE `SUDOERS`

```
# Usuario concreto, sin contraseña
nombre_usuario ALL=(ALL) NOPASSWD:ALL

# Cualquier miembro del grupo admin
%admin ALL=(ALL) ALL
```

El símbolo `%` indica un **grupo**. `NOPASSWD` evita pedir la contraseña al ejecutar `sudo`.

03

Identidad del sistema

Hostname y FQDN

Nombre del equipo en la red

HOSTNAME

- **Nombre corto** del equipo
- Identifica al sistema dentro de su red local
- Ejemplo: `sauron`

FQDN — *FULLY QUALIFIED DOMAIN NAME*

- **Nombre completo**, incluyendo el dominio
- Identifica al equipo de forma **inequívoca** en Internet
- Ejemplo: `sauron.mordor.com`
- Necesario para servicios como correo, web o certificados TLS

Archivos implicados

/ETC/HOSTNAME

- Contiene **únicamente el nombre corto** del equipo
- Es la fuente persistente del hostname tras un reinicio
- Una sola línea, sin dominio ni IP

/ETC/HOSTS

- Tabla de **resolución estática** local
- Permite que el propio equipo se resuelva sin depender del DNS
- La entrada para el equipo debe incluir **FQDN y hostname:**

```
127.0.1.1    sauron.mordor.com sauron
```

 La identidad del sistema requiere coherencia entre **ambos archivos**: si discrepan, distintos servicios verán nombres distintos.

hostnamectl : gestión centralizada del nombre

hostnamectl es la herramienta moderna (parte de **systemd**) para consultar y modificar la identidad del sistema **sin editar archivos a mano**.

PARA QUÉ SIRVE

- Cambia el hostname **de forma persistente**: actualiza `/etc/hostname` y aplica el cambio al sistema en ejecución, sin reiniciar
- Distingue entre varios tipos de nombre: *static*, *transient* y *pretty*
- Muestra información útil del equipo: kernel, arquitectura, ID de máquina, virtualización...
- Comandos típicos: `hostnamectl` (consulta) y `hostnamectl set-hostname sauron` (modificación)

 Aunque `hostnamectl` actualiza el hostname, **no modifica** `/etc/hosts`: ese archivo hay que ajustarlo aparte.

04

Configuración de red

Interfaces, direccionamiento y mecanismos de gestión

Conceptos básicos

Cada equipo conectado a una red necesita un conjunto mínimo de parámetros para comunicarse:

- **Interfaz de red:** dispositivo físico o virtual por el que circula el tráfico
- **Dirección IP:** identificador único dentro de la red
- **Máscara de red:** define el rango de direcciones de la subred
- **Puerta de enlace** (*gateway*): equipo al que se envía el tráfico hacia otras redes
- **Servidores DNS:** traducen nombres a direcciones IP

Direccionamiento estático vs dinámico

ESTÁTICO

- La IP se **asigna manualmente** y se mantiene fija
- Apropiado para **servidores** y equipos que ofrecen servicios
- Garantiza que la dirección no cambia tras reinicios
- Requiere planificación para evitar conflictos

DINÁMICO (DHCP)

- Un servidor **DHCP asigna** la configuración al arrancar
- La dirección puede **cambiar** entre concesiones
- Apropiado para **clientes** y equipos móviles
- Centraliza la administración de la red

Mecanismos de configuración en Linux

Linux ofrece **varias herramientas** para gestionar la red. La que se utiliza depende de la distribución y del rol del equipo (servidor, escritorio, portátil...).

HERRAMIENTA	ESTILO	USO TÍPICO
ifupdown	Tradicional, archivos planos	Debian clásico, servidores simples
NetworkManager	Dinámico, GUI + CLI	Escritorios, portátiles, Wi-Fi, VPN
systemd-networkd	Declarativo, integrado en systemd	Servidores modernos, contenedores
Netplan	Declarativo en YAML	Ubuntu (traduce a NM o systemd-networkd)

ifupdown

- Herramienta **tradicional** de Debian
- Archivos de configuración en `/etc/network/interfaces` y `/etc/network/interfaces.d/`
- Integrada con `systemd` mediante el servicio `networking.service`
- Sencilla y predecible: ideal para **servidores con configuración estable**

COMANDOS PRINCIPALES

```
ifup eth0          # Levanta una interfaz según la configuración
ifdown eth0        # La desactiva
ip a               # Muestra interfaces y direcciones IP
ip route           # Muestra la tabla de rutas
systemctl restart networking # Aplica cambios en interfaces
```

Ejemplo: `/etc/network/interfaces`

IP ESTÁTICA

```
auto eth0
iface eth0 inet static
    address 192.168.1.10
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 8.8.8.8 1.1.1.1
```

IP DINÁMICA (DHCP)

```
auto eth1
iface eth1 inet dhcp
```



La interfaz de **loopback** también se declara siempre: `auto lo` / `iface lo inet loopback`.

NetworkManager

- Pensada para **gestión dinámica** de la red
- Soporta **Wi-Fi, VPN, redes móviles** y conexiones cambiantes
- Habitual en **escritorios** y en distribuciones tipo Fedora
- Almacena las conexiones como *perfiles* en `/etc/NetworkManager/system-connections/`

COMANDOS PRINCIPALES

```
nmcli device status      # Estado de cada interfaz
nmcli connection show    # Lista de conexiones definidas
nmcli connection up <perfil> # Activa una conexión
nmtui                    # Interfaz interactiva en modo texto
nm-applet                 # Applet gráfico de escritorio
```

systemd-networkd

- **Forma parte de systemd:** integrada en el sistema base
- Configuración **declarativa** en archivos `.network` y `.link` dentro de `/etc/systemd/network/`
- Ligera y eficiente: pensada para **servidores** y entornos automatizados
- Combinada con `systemd-resolved` cubre red y DNS

COMANDOS PRINCIPALES

```
systemctl enable --now systemd-networkd    # Activar el servicio
systemctl restart systemd-networkd         # Aplicar cambios
networkctl                                  # Estado de las interfaces
networkctl status eth0                     # Detalle de una interfaz
```

Netplan

- Herramienta de **Ubuntu** (también disponible en otras distribuciones)
- Configuración declarativa en archivos **YAML** en `/etc/netplan/`
- No gestiona la red directamente: **traduce** la configuración a NetworkManager o systemd-networkd
- Unifica la sintaxis entre escritorio y servidor

COMANDOS PRINCIPALES

```
netplan generate    # Genera la configuración del backend
netplan apply       # Aplica los cambios
netplan try         # Prueba la configuración con rollback automático
```

¿Qué usa cada distribución?

DISTRIBUCIÓN	MECANISMO PRINCIPAL	ALTERNATIVA HABITUAL
Debian (servidor)	ifupdown	NetworkManager
Debian (escritorio)	NetworkManager	ifupdown
Ubuntu	Netplan → systemd-networkd	NetworkManager
Fedora / RHEL	NetworkManager	systemd-networkd

 Conviene **no mezclar** mecanismos en el mismo equipo: dos gestores intentando configurar la misma interfaz provocan conflictos imprevisibles.

Para profundizar — Configuración de red

Artículos del blog con explicaciones detalladas y ejemplos de cada herramienta:

- [Configuración de red en sistemas Linux — ifupdown](#)
- [Configuración de red en sistemas Linux — NetworkManager](#)
- [Configuración de red en sistemas Linux — systemd-networkd](#)
- [Configuración de red en sistemas Linux — Netplan](#)

05

Resolución de nombres

DNS, NSS y systemd-resolved

¿Qué es la resolución de nombres?

" Traducir un nombre legible (`www.ejemplo.com`) en la **dirección IP** que el equipo necesita para comunicarse.

IDEAS CLAVE

- Las redes funcionan con direcciones IP, las personas con nombres
- El **DNS** (*Domain Name System*) es el servicio jerárquico que realiza esa traducción a escala global
- En Linux la resolución no se delega únicamente al DNS: existen **fuentes locales** y un mecanismo que decide en qué orden consultarlas

NSS — Name Service Switch

NSS es la biblioteca del sistema que decide **dónde y en qué orden** se buscan distintos tipos de información: usuarios, grupos, hosts...

`/ETC/NSSWITCH.CONF`

- Para cada categoría especifica las **fuentes consultadas** y el orden
- En la línea `hosts:` define cómo se resuelven los nombres
- Las fuentes habituales son: archivos locales, DNS, mDNS, resolved...

 El orden típico es: **archivos locales primero, DNS después**. Permite sobrescribir un nombre puntualmente sin tocar el DNS global.

Ejemplos de la línea `hosts:` en `nsswitch.conf`

SERVIDOR SIN SYSTEMD-RESOLVED

```
hosts: files dns
```

- `/etc/hosts` primero, DNS después
- Configuración clásica con `ifupdown`

ESCRITORIO CON SYSTEMD-RESOLVED

```
hosts: files mdns4_minimal [NOTFOUND=return] resolve  
[!UNAVAIL=return] dns
```

- `files` → `mdns4_minimal` (`.local`) →
`resolve` (systemd-resolved) → `dns`

Archivos clásicos: hosts y resolv.conf

/ETC/HOSTS

- Tabla **estática** de nombres y direcciones IP
- Resolución **inmediata**, sin red
- Útil para nombres locales, entornos de prueba o sobreescrituras puntuales
- Se consulta antes que el DNS si así lo indica NSS

/ETC/RESOLV.CONF

- Lista de **servidores DNS** que se consultarán
- Puede incluir **dominios de búsqueda** y opciones avanzadas
- En sistemas modernos suele estar **gestionado automáticamente** por NetworkManager, systemd-resolved o el cliente DHCP
- Editarlo a mano puede ser **inútil** si lo regenera otro servicio

systemd-resolved

Servicio moderno que **centraliza** la resolución de nombres en el sistema.

QUÉ APORTA

- **Caché local** de respuestas → mejor rendimiento
- Servidor DNS local de reenvío en `127.0.0.53`
- Integración con **NSS** mediante módulos propios (`resolve` , `myhostname` , `mymachines`)
- Combina resolución estática (`/etc/hosts`), DNS remoto y nombres de contenedores

Módulos NSS que pueden aparecer en `hosts:`

MÓDULO	RESUELVE
<code>mdns4_minimal</code>	Nombres <code>.local</code> (multicast DNS)
<code>resolve</code>	Delega en systemd-resolved
<code>myhostname</code>	El propio hostname y <code>localhost</code> , sin red
<code>mymachines</code>	Contenedores/VMs locales (<code>systemd-nspawn</code>)

Comandos de `resolvectl`

```
resolvectl status           # Estado del servicio y DNS configurados
resolvectl query www.ejemplo.com # Consulta usando systemd-resolved
resolvectl dns eth0 1.1.1.1   # Establece servidores DNS para una interfaz
resolvectl flush-caches      # Vacía la caché de resolución
```

 `resolvectl` usa el flujo del propio sistema (caché incluida), por lo que sus respuestas reflejan lo que verá una aplicación real.

Herramientas de consulta: cuidado con lo que prueban

No todas las herramientas siguen el mismo camino que el sistema operativo cuando una aplicación normal resuelve un nombre.

- **Consultas directas al DNS:** `dig`, `host`, `nslookup`
No respetan el orden de NSS — diagnostican el DNS en sí mismo
- **Consultas que respetan NSS:** `getent hosts`, `getent ahosts`
Reproducen el camino real de una aplicación

⚠ Si `dig` resuelve un nombre pero `ping` no, lo más probable es que el problema esté en NSS o en `/etc/hosts`, no en el DNS.

Comandos de consulta DNS

DIRECTOS AL DNS

```
dig ejemplo.com  
host ejemplo.com  
nslookup ejemplo.com
```

RESPETANDO NSS

```
getent hosts ejemplo.com  
getent ahosts ejemplo.com
```

Para profundizar — Resolución de nombres

- [Resolución de nombres de dominios en sistemas Linux](#)

 El artículo desarrolla en detalle el funcionamiento de NSS, `/etc/hosts`, `/etc/resolv.conf` y `systemd-resolved`, con ejemplos prácticos.

06

Router Linux, SNAT y DNAT

Enrutamiento y traducción de direcciones con iptables y nftables

¿Para qué se usa un router Linux?

Un equipo Linux puede actuar como **router** para conectar dos o más redes y permitir el enrutamiento de paquetes entre ellas. Es habitual en redes domésticas, laboratorios docentes o firewalls personalizados.

FUNCIONES QUE PUEDE REALIZAR

- **NAT** — traducción de direcciones de red
- **Filtrado de paquetes** — control del tráfico permitido
- **Redirección de puertos** — exponer servicios internos al exterior
- **Compartir conexión a Internet** — dar salida a una red privada

 Linux dispone de todo lo necesario en el propio kernel: sólo hay que **activar el reenvío** y definir las reglas adecuadas con `iptables`.

Habilitar el reenvío de IPs (IP Forwarding)

Por defecto el kernel **no reenvía** paquetes entre interfaces. Para que un equipo actúe como router hay que activarlo.

ACTIVACIÓN TEMPORAL

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

Se pierde tras reiniciar el sistema.

ACTIVACIÓN PERSISTENTE (DEBIAN 13)

En **Debian 13** ya no se proporciona `/etc/sysctl.conf`: la configuración se organiza de forma modular en `etc/sysctl.d/`.

```
echo "net.ipv4.ip_forward = 1" > /etc/sysctl.d/99
```

Aplicar los cambios:

```
sysctl --system
```

nftables: el sucesor de iptables

`nftables` es el framework de filtrado y NAT del kernel Linux que **sustituye** a `iptables`, `ip6tables`, `arptables` y `ebtables` por un único subsistema.

- Unifica **IPv4 e IPv6** en una sola sintaxis: ya no hay que duplicar reglas en `ip6tables`
- Las reglas se agrupan en **tablas y cadenas definidas por el usuario**, cargadas de forma **atómica**
- En Debian/Ubuntu recientes, el propio comando `iptables` es en realidad `iptables-nft`: por debajo traduce las reglas a nftables
- `nft` es la herramienta **nativa**, con sintaxis declarativa propia

 `iptables` sigue funcionando gracias a esa capa de compatibilidad, pero **nftables es el sustituto recomendado** para configuraciones nuevas.

SNAT — Source NAT

Permite que una red local con **direcciones privadas** salga a Internet usando la **IP pública** del router. Cambia la **IP de origen** de los paquetes salientes.

IPTABLES

```
iptables -t nat -A POSTROUTING \  
-o eth0 -s 192.168.0.0/24 \  
-j SNAT --to-source 192.0.2.1
```

NFTABLES

```
table ip nat {  
    chain postrouting {  
        type nat hook postrouting priority 100;  
        ip saddr 192.168.0.0/24 oifname "eth0" snat t  
    }  
}
```

 `192.0.2.1` es la IP pública fija del router; iptables añade una regla, nftables la declara dentro de la cadena `postrouting`.

SNAT con IP dinámica: MASQUERADE

Cuando la IP pública del router **no es fija** (por ejemplo, asignada por DHCP del proveedor), conviene usar `MASQUERADE` / `masquerade` en lugar de indicar la IP.

IPTABLES

```
iptables -t nat -A POSTROUTING \  
-o eth0 -s 192.168.0.0/24 \  
-j MASQUERADE
```

NFTABLES

```
table ip nat {  
    chain postrouting {  
        type nat hook postrouting priority 100;  
        ip saddr 192.168.0.0/24 oifname "eth0" masquerade  
    }  
}
```

 Toma automáticamente la IP de la interfaz de salida: no hay que especificarla, ideal cuando puede cambiar.

DNAT — Destination NAT

Se usa para **redirigir tráfico entrante** desde el exterior hacia una máquina de la red interna. Cambia la **IP de destino** de los paquetes.

IPTABLES

```
iptables -t nat -A PREROUTING \  
-i eth0 -p tcp --dport 80 \  
-j DNAT --to-destination 192.168.1.100:80
```

NFTABLES

```
table ip nat {  
    chain prerouting {  
        type nat hook prerouting priority -100;  
        iifname "eth0" tcp dport 80 dnat to 192.168.1.100:80;  
    }  
}
```

 Se aplica en `PREROUTING` porque la decisión de redirigir se toma **antes** de enrutar el paquete.

SNAT vs DNAT: comparación

SNAT — SALIDA

- Cambia la **IP de origen**
- Cadena `POSTROUTING`
- Sentido: **red interna → Internet**
- Comparte una IP pública entre varios equipos
- Caso típico: dar Internet a una LAN

DNAT — ENTRADA

- Cambia la **IP de destino**
- Cadena `PREROUTING`
- Sentido: **Internet → red interna**
- Publica un servicio interno hacia el exterior
- Caso típico: redirección de puertos a un servidor

Hacer las reglas persistentes

Ni las reglas de `iptables` ni las de `nftables` **sobreviven a un reinicio** por defecto. Hay que guardarlas.

IPTABLES (`IPTABLES-PERSISTENT`)

```
iptables-save > /etc/iptables/rules.v4
```

Equivalente IPv6 en `/etc/iptables/rules.v6`.

NFTABLES

```
nft list ruleset > /etc/nftables.conf  
systemctl enable --now nftables
```

El servicio carga ese fichero al arrancar.

 Después de modificar reglas, no olvides **volver a guardarlas**: si no, los cambios se perderán en el próximo reinicio.

Para profundizar — nftables

- [nftables: cortafuegos perimetral y NAT](#)

 El artículo desarrolla la configuración de un cortafuegos perimetral completo con `nftables`, incluyendo SNAT y DNAT.

SRI · UNIDAD 1 — CONFIGURACIÓN INICIAL DE UN SERVIDOR LINUX

¡Gracias!

Configuración inicial

 José Domingo Muñoz

 IES Gonzalo Nazareno · Dos Hermanas

 SRI